# JWT Auth App

## Technical Design & Implementation Report

Flutter · Dart · JWT · ReqRes API · Secure Storage

Version 1.0.0+1 · June 3, 2026

## 1. Project Overview

The JWT Auth App is a Flutter application that demonstrates a complete JWT-based authentication system against the ReqRes.in demo API. It covers the full lifecycle: login, registration, persistent secure token storage, protected routing, and auto-login on app restart.

| | |
|---|---|
| **App Name** | jwt_auth_app |
| **Version** | 1.0.0+1 |
| **Flutter SDK** | ^3.11.5 (Dart) |
| **API Backend** | ReqRes.in (https://reqres.in) |
| **Primary Packages** | dio ^5.9.2, flutter_secure_storage ^10.3.1 |
| **Architecture** | Repository + ChangeNotifier (Controller pattern) |
| **Target Platforms** | Android, iOS (Flutter cross-platform) |

### Learning Objectives Addressed

- Understand JWT authentication — token structure, access vs. refresh tokens
- API integration via the Dio HTTP client with interceptors
- Complete authentication flow — login, register, auto-login, logout
- Secure token storage using flutter_secure_storage
- Protected routing with an auth gate widget

## 2. Architecture & Layer Design

The app follows a clean three-layer architecture that separates concerns between data access, business logic, and UI.

**Layer Diagram**

*UI Layer → AuthController (ChangeNotifier) → AuthRepository → ApiService / AuthStorage*

### 2.1 Data Layer

- AuthStorage — wraps FlutterSecureStorage; handles read, write, and delete of the serialized session JSON under the key jwt_auth_session.
- ApiService — a Dio-based HTTP client targeting https://reqres.in. Attaches an x-api-key header globally and injects the Bearer token via a Dio interceptor on every request that carries attachAuth: true.

### 2.2 Domain / Repository Layer

- AuthRepository — orchestrates the full auth lifecycle. Calls ApiService, builds AuthSession objects, persists them via AuthStorage, and keeps an in-memory reference to the active session.
- bootstrap() — reads persisted session on cold start and restores state without requiring the user to log in again.
- login() / register() — validate demo credentials, call the API, build a session, and persist it.
- logout() — clears both the in-memory session and the secure store.

### 2.3 Presentation Layer

- AuthController — a ChangeNotifier that delegates to AuthRepository and exposes observable flags (isBootstrapping, isSubmitting, isAuthenticated).
- AuthScope — an InheritedNotifier that makes the controller available anywhere in the widget tree.
- AuthGate — reactive widget that switches between SplashScreen, LoginScreen, and HomeScreen based on controller state.

## 3. Key Classes & Files

| Class / File | Type | Responsibility |
|---|---|---|
| `main.dart` | Entry point | Creates JwtAuthApp, owns AuthController, calls bootstrap() |
| `auth_repository.dart` | Data model | AuthUser — immutable user model with displayName helper |
| `auth_repository.dart` | Data model | AuthSession — pairs AuthUser with accessToken & refreshToken |

| Class / File | Type | Responsibility |
|---|---|---|
| auth_repository.dart | Exception | AuthApiException — typed error carrying message + statusCode |
| auth_repository.dart | Storage | AuthStorage — secure read/write/delete of the session JSON |
| auth_repository.dart | HTTP client | ApiService — Dio wrapper; login(), register(), Bearer interceptor |
| auth_repository.dart | Repository | AuthRepository — auth lifecycle orchestrator |
| auth_repository.dart | Controller | AuthController — ChangeNotifier; bridges repo to UI |
| auth_screens.dart | Widget | AuthScope — InheritedNotifier for DI of AuthController |
| auth_screens.dart | Widget | AuthGate — route guard switching on controller state |
| auth_screens.dart | Screen | SplashScreen — shown during bootstrap |
| auth_screens.dart | Screen | LoginScreen — form with email/password; demo credentials card |
| auth_screens.dart | Screen | RegisterScreen — 6-field form; auto-navigates home after success |
| auth_screens.dart | Screen | HomeScreen — protected screen; shows profile, token preview, flow info |

# 4. Authentication Flow

## 4.1 App Startup (bootstrap)

- WidgetsFlutterBinding.ensureInitialized() called before runApp.
- AuthController.bootstrap() is triggered in initState of JwtAuthApp.
- AuthRepository.bootstrap() reads the secure store; if a valid accessToken exists the session is restored and the user proceeds directly to HomeScreen.
- During this check, SplashScreen is displayed (isBootstrapping == true).

## 4.2 Login Flow

- User enters credentials on LoginScreen and taps Sign in.
- Form validation runs (non-empty email, password >= 6 chars).
- AuthController.login() sets isSubmitting = true and calls AuthRepository.login().
- AuthRepository hard-validates against the ReqRes demo credentials (eve.holt@reqres.in / cityslicka) and throws AuthApiException for any mismatch.
- On success, ApiService.login() POST /api/login is called with the Dio client.
- The returned token is wrapped in AuthSession, persisted via AuthStorage, and held in memory.
- AuthController notifies listeners; AuthGate rebuilds and shows HomeScreen.

### 4.3 Registration Flow

- User navigates to RegisterScreen via the "Create a new account" button.
- Form collects first name, last name, username, email, password, and confirm password.
- AuthRepository validates against the ReqRes demo credentials (eve.holt@reqres.in / pistol).
- POST /api/register is called; the returned token is stored and the user is auto-logged in.
- On success, Navigator.popUntil(isFirst) closes RegisterScreen and AuthGate shows HomeScreen.

### 4.4 Logout Flow

- User taps the logout icon in the HomeScreen AppBar.
- AuthController.logout() is called asynchronously with loading state.
- AuthRepository.logout() sets _currentSession = null, clears ApiService session reference, and calls AuthStorage.clearSession() (deletes the secure storage entry).
- AuthController notifies listeners; AuthGate rebuilds and shows LoginScreen.

## 5. Token Management & Security

### 5.1 Storage

flutter_secure_storage encrypts data at rest using the platform keychain/keystore (iOS Keychain, Android Keystore). The entire AuthSession object is serialized to JSON and stored under a single key:

```
static const String _sessionKey = 'jwt_auth_session';
```

The stored JSON schema is:

```
{ "user": { ... }, "accessToken": "...", "refreshToken": "..." }
```

### 5.2 Bearer Token Injection

ApiService installs a Dio InterceptorsWrapper during construction. On every outgoing request it checks:

- Whether options.extra['attachAuth'] is true (default for all API calls except login/register).
- Whether a non-empty accessToken exists in the active session.

If both conditions are met it sets the Authorization: Bearer <token> header automatically — no manual work required at the call site.

### 5.3 Refresh Token

The ReqRes demo API does not issue refresh tokens. The current implementation stores an empty refreshToken field in AuthSession as a placeholder. A production system should replace AuthRepository.bootstrap() with a token-refresh call when the access token has expired.

### 5.4 Error Handling

AuthApiException is a typed exception carrying a human-readable message and optional HTTP status code. ApiService._toAuthException() maps DioException types to user-friendly strings:

| | |
|---|---|
| **connectionTimeout** | Connection timed out. Try again. |
| **sendTimeout** | Request timed out. Try again. |
| **receiveTimeout** | Server took too long to respond. |
| **connectionError** | No internet connection detected. |
| **HTTP 401** | Session expired. Sign in again. |
| **Other** | Authentication request failed. |

## 6. UI Screens

### 6.1 SplashScreen

- Displayed only during bootstrap (isBootstrapping == true).
- Shows the app name, a shield icon, and a CircularProgressIndicator.adaptive().
- Replaced immediately by LoginScreen or HomeScreen once bootstrap completes.

### 6.2 LoginScreen

- Pre-filled with demo credentials for ease of testing.
- Email (TextInputType.emailAddress) and obscured password fields.
- FilledButton disabled and replaced with a spinner while isSubmitting.
- A _DemoCredentialsCard widget at the bottom shows the valid test credentials.
- AuthApiException messages are surfaced via SnackBar.

### 6.3 RegisterScreen

- Six fields: first name, last name, username, email, password, confirm password.
- First/last name fields displayed side-by-side using a Row with two Expanded children.
- Password confirm validator compares against the password field at submission time.

- After successful registration, popUntil(isFirst) returns to the initial route, which AuthGate now renders as HomeScreen.

### 6.4 HomeScreen (Protected)

- Only reachable when isAuthenticated == true.
- Displays a profile card with the user's display name, email, and status pills.
- Shows a truncated preview of the stored access token (first 14 + last 10 characters).
- Lists the three authentication-flow bullets explaining what happened behind the scenes.
- Logout icon in the AppBar triggers AuthController.logout().

### 6.5 Design System

- Material 3 theme with ColorScheme seeded from teal (#0F766E).
- Dark gradient backdrop (0F172A → 134E4A → 0B1120) on auth screens with decorative orbs.
- Rounded cards (28 px radius), filled inputs (18 px radius), no borders on text fields.
- Scaffold background: #F4F7FB (light blue-grey).

## 7. Dependencies

| Package | Version | Purpose |
|---|---|---|
| `dio` | ^5.9.2 | HTTP client with interceptors, timeout config, typed exceptions |
| `flutter_secure_storage` | ^10.3.1 | Keychain/Keystore-backed encrypted key-value storage |
| `cupertino_icons` | ^1.0.8 | iOS-style icon set for adaptive widgets |
| `flutter_lints` | ^6.0.0 (dev) | Static analysis lint rules |

## 8. Deliverables & Status

| Deliverable | Status | Location |
|---|---|---|
| Authentication system (login + register + logout) | ✓Done | auth_repository.dart |
| Login screen with validation | ✓Done | auth_screens.dart |
| Register screen with 6 fields | ✓Done | auth_screens.dart |
| JWT token stored in secure storage | ✓Done | AuthStorage class |
| Auto-login on app restart (bootstrap) | ✓Done | AuthRepository.bootstrap() |

| Deliverable | Status | Location |
|---|---|---|
| Protected HomeScreen (auth gate) | ✓Done | AuthGate widget |
| Bearer token injected on all API requests | ✓Done | Dio interceptor in ApiService |
| Splash screen while checking auth | ✓Done | SplashScreen widget |
| Error handling with user-facing messages | ✓Done | AuthApiException + SnackBar |
| Refresh token logic | □ Placeholder | AuthSession.refreshToken = "" |

# 9. Recommended Improvements

## 9.1 Refresh Token Implementation

- Replace the empty refreshToken field with real logic: store the refresh token, and in ApiService add a response interceptor that catches 401 responses, calls a refresh endpoint, updates the session, and retries the original request.

## 9.2 Real API Integration

- The hard-coded credential check inside AuthRepository is a ReqRes-specific workaround. Remove it and point BaseOptions.baseUrl at a production backend.
- Add a GET /api/user/{id} call to populate AuthUser with real server data.

## 9.3 State Management

- For larger apps, replace AuthController + InheritedNotifier with Riverpod or BLoC for better testability and scalability.

## 9.4 Testing

- Add unit tests for AuthRepository using a mock ApiService and mock AuthStorage.
- Add widget tests for LoginScreen and RegisterScreen using a mock AuthController.
- Add integration tests that simulate the full login → home → logout cycle.

## 9.5 Security Hardening

- Add certificate pinning via dio_certificate_pinning to prevent MITM attacks.
- Implement session timeout: check token expiry (JWT exp claim) in bootstrap() and force re-login when expired.
- Obfuscate the API key — do not store it in plain source code.

# 10. Auth Flow Summary (README)

The following summarizes the authentication flow as it would appear in the project README:

### Cold Start

bootstrap() reads flutter_secure_storage. If a valid token exists the user lands on HomeScreen without re-entering credentials.

### Login

POST /api/login with {email, password}. The API returns a token which is wrapped in AuthSession and saved securely. The app navigates to HomeScreen.

### Register

POST /api/register with {email, password}. Returns a token; user is auto-logged in without a separate login step.

### API Requests

Dio interceptor attaches Authorization: Bearer <token> to every request automatically once a session is active.

### Logout

Session cleared from memory and secure storage. AuthGate rebuilds and shows LoginScreen.

*End of Report*